OPTIONS REPORT · 18.10.2026

How should jarvis reindex after a pull or edit burst?

Recommended: Option A — Carry-forward with closure invalidation (confidence: medium).

Question and context

How should jarvis avoid re-running the whole-project SCIP indexer on every reindex when only a small diff changed?

Every jarvis index run rebuilds from scratch: only the published commit sha is compared to detect staleness, and the SCIP compiler indexer then re-runs over the whole repo. Measured this session on this small Python repo: scip-python takes 11.8s of a roughly 15s non-semantic reindex (zoekt 0.7s, convert 0.3s), and on the Java and TypeScript repos in the local registry the indexer runs take minutes. Upstream indexers offer no partial or incremental mode, and the scip CLI has no merge command. The syntax baseline and the semantic vector stage are already incremental; the SCIP stage and the run trigger are not.

Constraints

- Publishing stays atomic: immutable index-<sha>-<generation>.db plus a current pointer flip; readers never see partial state
- Navigation answers stay honest: reused SCIP data must be labeled (scip_state, freshness), never served as fresh
- No forked or patched indexer binaries; indexers ship as pinned prebuilt executables
- A skipped or degraded SCIP stage never fails the run (existing stage-3 contract)

Decision drivers

- SCIP is the dominant reindex cost once syntax and vectors are incremental
- Correctness before speed: a silently stale findReferences is worse than a slow reindex
- jarvis watch usability: today a 5s debounce triggers the full pipeline, which is why watch gets turned off
- Blast radius: how much of the pipeline, registry status vocabulary, and server surface must change

Options

OPTION A · RECOMMENDED

Carry-forward with closure invalidation

Compute changed files from the capture hashes, use the previous snapshot's own mentions data to find every file that references symbols defined in changed files (the reverse-symbol closure), copy those SCIP rows forward untouched, and let invalidated files fall back to syntax-only navigation under a new scip_state like carried@<old-sha>. A full SCIP run still happens on explicit indexRepo calls or when the closure exceeds a threshold.

PROS

- watch and edit-burst reindexes skip the indexer: about 12s down to about 2s on this repo, minutes to seconds on the Java/TS repos
- unchanged files keep precise navigation and invalidated files degrade exactly like today's degraded state, so the honesty machinery (scip_state, per-file capability errors) already exists
- closure cost verified on a real snapshot: milliseconds per query, and leaf-file diffs invalidate 2-3 of 30 documents; core-file diffs self-limit toward full reindex

CONS

- not bit-identical to a full run: a new competing definition

OPTION B

Same-HEAD no-op short-circuit

One guard at the top of the pipeline: when the published commit sha equals HEAD and the source hash is unchanged, publish nothing and return success. Optionally let zoekt keep its own shard when nothing moved.

PROS

- smallest possible change: a single guard in _index_repo_locked, no snapshot semantics, no new status values
- kills the measured waste of identical re-runs: a warm same-HEAD run still costs 53s today (SCIP 11.8s, zoekt, semantic model load)
- zero correctness risk because nothing is reused across different content

CONS

- does nothing for the actual pull case: HEAD moved means the full pipeline runs, so the 12s-to-minutes SCIP cost stays
- watch runs on uncommitted edits change the source hash and still pay the full SCIP run
- the skip must not flip registry statuses or clear prior SCIP failure records, which is subtle bookkeeping for a small win

BEST WHEN

OPTION C

Partial indexing upstream

Patch or wrap the indexers for file-subset emission, or index module by module and merge the results. Build the merge tooling the scip CLI currently lacks.

PROS

- the only option where a code diff produces truly fresh SCIP for changed files without a whole-repo compiler pass
- per-document emission from the same compiler can be bit-identical to a full run

CONS

- forks every indexer jarvis supports: python, typescript, java and swift each wrap a different compiler or build system where partial runs are ill-defined
- jarvis's distribution model (pinned prebuilt binaries via Homebrew and setup.sh) would carry those patches forever
- an order of magnitude more work than A, with no upstream merge command to build on

BEST WHEN

Incremental SCIP becomes a protocol feature owned upstream, so jarvis consumes it instead of patching it.

in a changed file can silently re-point references in files outside the closure until the next full run
- a new scip_state value plus a snapshot merge path that server.py, getIndexStatus, and the dashboard must all understand
- carry-forward needs a previous generation with usable SCIP data; after a failed or unavailable run it degrades to a full re-run anyway

BEST WHEN

Watch-mode latency and big-repo reindex time matter more than bit-identical SCIP freshness, and periodic full refreshes are acceptable.

The dominant waste is repeated identical runs (dashboards, CI, agent re-calls) and full SCIP after a real pull is acceptable.

Recommendation

Option A — Carry-forward with closure invalidation

A is the only option that attacks the dominant cost for the real pull and watch cases while staying inside jarvis's own process boundary: no forked binaries, and stale data is labeled through the same degradation machinery the server already exposes. B's no-op win is mostly subsumed (an unchanged diff yields an empty closure, so the indexer is skipped anyway), and C fails the distribution constraint.

TRADE-OFFS ACCEPTED

- a small class of stale answers (resolution flips) until the next full SCIP run
- one new scip_state value and a snapshot merge path to maintain
- no-op runs still pay capture, syntax and zoekt (a few seconds) rather than B's full skip

REVISIT WHEN

- upstream scip or any indexer ships partial or incremental emission
- closure measurements on the large Java/TS repos show core-file diffs invalidate nearly everything, erasing the win
- users report stale-navigation incidents from carried generations in practice

Confidence: **medium**

Evidence

Claim	Kind	Ref
scip-python and scip-typescript CLIs expose no incremental, partial, or merge flags	source	scip-python --help and scip-typescript --help output, checked 2026-10-18
The scip CLI has no merge command; expt-convert is one-shot whole-index to SQLite	source	scip --help output (v0.9.0-dev), checked 2026-10-18
The converted SCIP schema is per-document keyed (documents.relative_path UNIQUE; chunks, mentions, defn_enclosing_ranges reference document ids), so per-file copy-forward is structurally possible	source	.schema dump of ~/.jarvis/scip/_/polaris-codebase-registry/_/index-73db74ae....db
Reverse-symbol closure on a real 30-document snapshot is millisecond-cheap: 2/8/29/64 defined symbols invalidate 3/2/8/9 documents respectively	source	SQL closure queries on the same snapshot, this session
Stage costs measured on this repo: scip-python 11.8s, expt-convert 0.26s, zoekt 0.73s; full run 107.6s cold and 53.1s warm same-HEAD	source	timed commands run 2026-10-18 (time jarvis index ., time scip-python index, time zoekt-git-index)
The syntax baseline already reuses per-file rows from the previous snapshot when file_hash and grammar_identity match	source	src/jarvis/syntax_index.py:524-531
Zoekt is deliberately forced non-incremental and upstream incrementality is repo-granularity, which a pull defeats anyway	source	src/jarvis/index_cli.py:661-675
Watch is a 5s debounce into the full pipeline, and the known gap is SCIP/Zoekt being all-or-nothing	source	docs/kilo-vs-jarvis-indexing.md:28,58,79
A new competing definition can re-point references in files outside the closure without changing those files (resolution flip)	assumption	—
Periodic full SCIP refreshes are acceptable to jarvis users	assumption	—

Open questions

- What triggers a full SCIP refresh: explicit indexRepo only, N carried generations, or a closure-size threshold?
- Should getIndexStatus expose a distinct freshness field for carried generations?
- Should watch default to carry-forward while explicit indexRepo calls always run full SCIP?

Next step

Approve option A and start the architectural design flow (clarifying questions, then a written spec).